

Java'da Decision Making ve for Loops

if-else, switch, for ve for-each

Java Programlama · Dr. Öğr. Üyesi Baki Gökgöz · Gümüşhane Üniversitesi

Bu Derste Neler Öğreneceğiz?



if Statements

Simple if, if-else, if-else-if ladder, nested if ve ternary ile kısa karar yapıları.



switch

Bir değeri birçok sabitle karşılaştırmak; fall-through, String/enum switch ve modern switch expression.



for Loop

Sayaç kontrollü tekrar, nested for, labeled break ve klasik algoritmalar.



for-each Loop

Array ve Collection elemanlarını index kullanmadan gezmek; sınırlamaları.

Bölüm 1

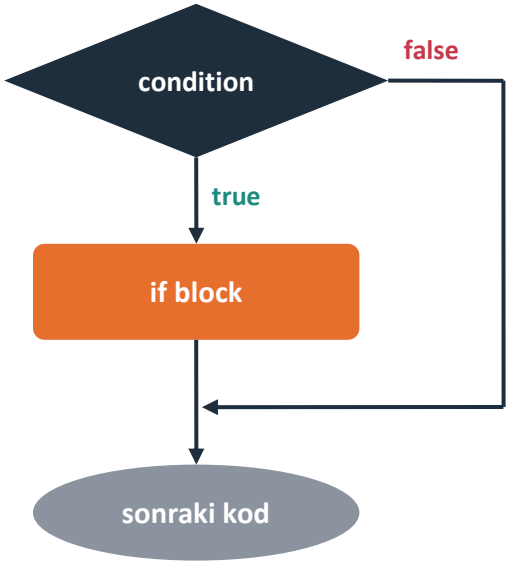


if Statements

Koşula göre farklı kod çalıştırmak

Decision Making: if Statement

- Program bir **koşulu (condition)** değerlendirir ve sonuca göre hangi kodun çalışacağına karar verir.
- Koşul mutlaka **boolean** sonuç üretmelidir: `true` veya `false`.
- Java'da 4 tür if yapısı vardır:
  - simple **if**
  - **if-else**
  - **if-else-if** ladder
  - **nested if**
- Tek satırlık body'de `{ }` zorunlu değildir, ama **her zaman** kullanılması önerilir.



Simple if Statement

```java
int age = 20;
if (age > 18) {
    System.out.println("Yaş 18'den büyük");
}

int temperature = 35;
if (temperature > 30) {
    System.out.println("Hava sıcak, su için!");
}
if (temperature < 0) {  // false: atlanır
    System.out.println("Don uyarısı!");
}
System.out.println("Program devam ediyor");
```

Çıktı (Output)

```
Yaş 18'den büyük
Hava sıcak, su için!
Program devam ediyor
```

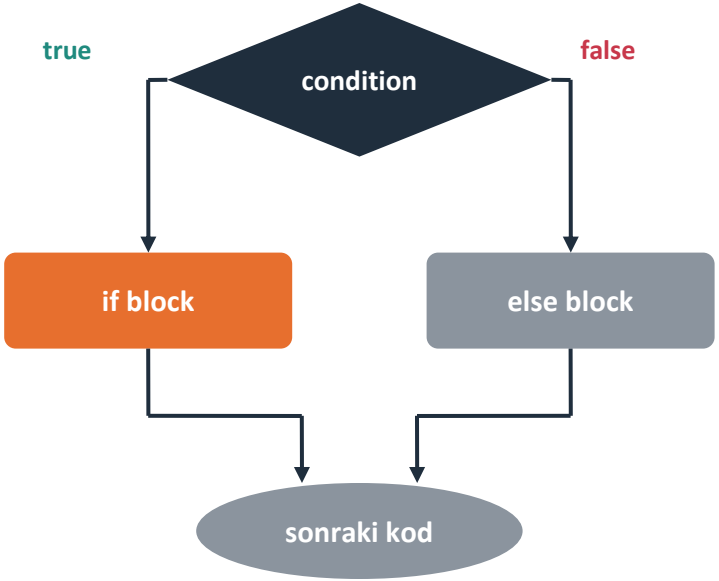
- Koşul **true** ise block çalışır; **false** ise block atlanır ve program bir sonraki satırdan devam eder.
- Kullanım alanları: doğrulama (validation), uyarılar, tek yönlü kontroller.

if-else Statement

```java
int number = 13;
// 2'ye tam bölünüyor mu?
if (number % 2 == 0) {
    System.out.println(number + " çift sayıdır");
} else {
    System.out.println(number + " tek sayıdır");
}
```

Çıktı (Output)

```
13 tek sayıdır
```



Örnek: Leap Year (Artık Yıl) Kontrolü

```java
int[] years = {2020, 2023, 1900, 2000};
for (int year : years) {
    boolean leap = (year % 4 == 0 && year % 100 != 0)
                 || (year % 400 == 0);
    if (leap) {
        System.out.println(year + " -> LEAP YEAR");
    } else {
        System.out.println(year + " -> COMMON YEAR");
    }
}
```

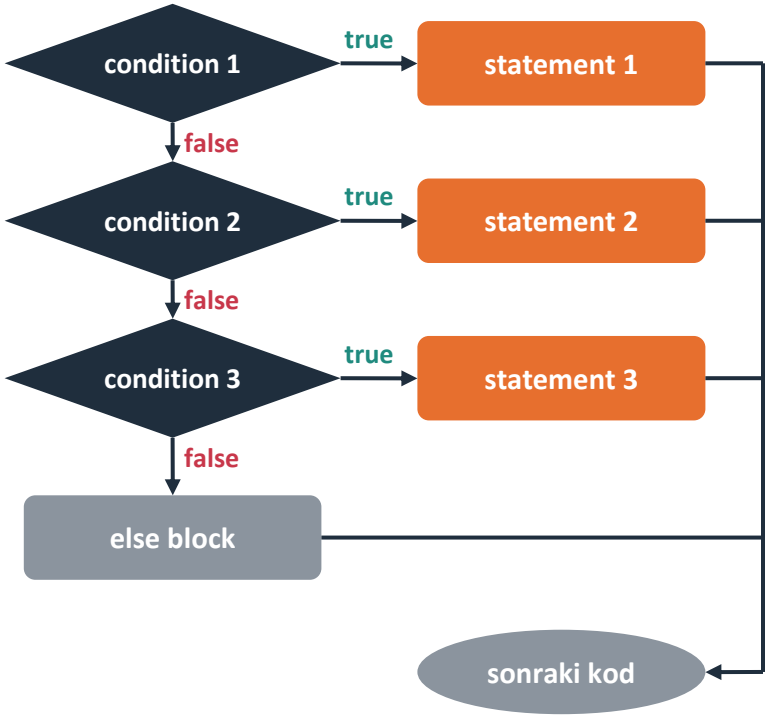
Çıktı (Output)

```
2020 -> LEAP YEAR
2023 -> COMMON YEAR
1900 -> COMMON YEAR
2000 -> LEAP YEAR
```

- **Kural:** 4'e bölünüp 100'e bölünmeyen **veya** 400'e bölünen yıllar artık yıldır.
- 1900: 4 ve 100'e bölünür, 400'e bölünmez → artık yıl **değil**.
- Karmaşık koşulu bir boolean variable'a atamak okunabilirliği artırır.

if-else-if Ladder

- Birden fazla koşulu **sırayla** kontrol eder.
- Koşullar yukarıdan aşağıya değerlendirilir; **ilk true** olan block çalışır, geri kalanı atlanır.
- Hiçbiri true değilse (varsa) en sondaki `else` çalışır.
- Tipik kullanım: not sistemi, vergi dilimleri, menü seçimleri.



if-else-if Ladder: Not Sistemi

```java
int marks = 65;
if (marks < 0 || marks > 100) {
    System.out.println("Geçersiz not!");
} else if (marks >= 90) {
    System.out.println("A+ grade");
} else if (marks >= 80) {
    System.out.println("A grade");
} else if (marks >= 70) {
    System.out.println("B grade");
} else if (marks >= 60) {
    System.out.println("C grade");
} else if (marks >= 50) {
    System.out.println("D grade");
} else {
    System.out.println("fail");
}
```

Çıktı (Output)

```
C grade
```

- Önce geçersiz değerler elenir, sonra büyükten küçüğe kontrol edilir.
- Bu sıralama sayesinde **`marks >= 60 && marks < 70`** gibi tekrarlı koşullara gerek kalmaz.
- Kaynaktaki örnekte **`marks < 100`** yazıldığı için 100 puan "Invalid" çıkıyordu; burada düzeltildi.

Ladder'da Koşul Sırası Önemlidir

✗ Yanlış sıra

```
int marks = 95;
if (marks >= 50) {
    System.out.println("D grade");
} else if (marks >= 90) {
    System.out.println("A+ grade");
}
```

Çıktı (Output)

```
D grade
```

- 95 ≥ 50 true → ilk block çalışır, **>= 90** hiç kontrol edilmez.
- Mantık hatası (logic error): derleyici uyarmaz!

✓ Doğru sıra: en kısıtlayıcı koşul önce

```
int marks = 95;
if (marks >= 90) {
    System.out.println("A+ grade");
} else if (marks >= 50) {
    System.out.println("D grade");
}
```

Çıktı (Output)

```
A+ grade
```

- Aralıklı koşullarda büyükten küçüğe (veya tersi) tutarlı sıra kullanın.

Örnek: Positive, Negative veya Zero

```java
int[] numbers = {-13, 0, 42};
for (int number : numbers) {
    if (number > 0) {
        System.out.println(number + ": POSITIVE");
    } else if (number < 0) {
        System.out.println(number + ": NEGATIVE");
    } else {
        System.out.println(number + ": ZERO");
    }
}
```

Çıktı (Output)

```
-13: NEGATIVE
0: ZERO
42: POSITIVE
```

- Üç durumdan **tam olarak biri** gerçekleşir; son durum için koşul yazmaya gerek yoktur (`else`).
- Test ederken her dalı (branch) en az bir kez çalıştıracak değerler seçin: negatif, sıfır, pozitif.

Nested if Statement

```java
int age = 25;
int weight = 48;

if (age >= 18) {
    if (weight > 50) {
        System.out.println("Kan bağışı yapabilirsiniz");
    } else {
        System.out.println("Kilo yetersiz (50 kg üstü)");
    }
} else {
    System.out.println("Yaş en az 18 olmalı");
}
```

Çıktı (Output)

```
Kilo yetersiz (50 kg üstü)
```

- İçteki if, yalnızca dıştaki koşul **true** ise kontrol edilir.
- Mesaj ile koşul tutarlı olmalı: kaynakta `age >= 18` kontrol edilip "18'den büyük olmalı" yazıyordu.
- 3+ seviye iç içe if okunabilirliği düşürür; `&&` veya erken `return` tercih edin.

Nested if mi, && mi?

Nested if

```java
int age = 20, weight = 80;
if (age >= 18) {
    if (weight > 50) {
        System.out.println("Uygun");
    }
}
```

Çıktı (Output)

```
Uygun
```

- Her koşul için **ayrı mesaj/işlem** gerekiyorsa uygundur (önceki slayttaki gibi).

Logical AND (&&)

```java
int age = 20, weight = 80;
if (age >= 18 && weight > 50) {
    System.out.println("Uygun");
}
```

Çıktı (Output)

```
Uygun
```

- Tek bir sonuç varsa daha kısa ve okunaklıdır.
- Short-circuit sayesinde ilk koşul false ise ikinci kontrol edilmez.

if-else Yerine Ternary Operator

if-else

```java
int number = 13;
String output;
if (number % 2 == 0) {
    output = "çift";
} else {
    output = "tek";
}
System.out.println(output);
```

Çıktı (Output)

```
tek
```

Ternary (? :)

```java
int number = 13;
String output = (number % 2 == 0)
        ? "çift" : "tek";
System.out.println(output);
```

Çıktı (Output)

```
tek
```

- Basit **değer seçimi** için idealdir; çok satırlı işlerde if-else.

if ile İlgili Sık Yapılan Hatalar

```java
int x = 3;
if (x > 5);                    // DİKKAT: boş statement!
{
    System.out.println("A: Her zaman çalışır");
}

if (x > 5)
    System.out.println("B: koşula bağlı");
    System.out.println("C: if'e bağlı DEĞİL");

// if (x = 5) { }    -> compile error: int boolean değil
// if (x) { }        -> compile error (C'den farklı!)
```

Çıktı (Output)

```
A: Her zaman çalışır
C: if'e bağlı DEĞİL
```

- `if (...);` → noktalı virgül if'in body'sidir; alttaki block her zaman çalışır.
- `{ }` olmadan sadece **ilk satır** if'e bağlıdır; girinti (indentation) anlam taşımaz.
- Java'da koşul **boolean** olmalıdır; `0/1` veya atama kullanılamaz.

Bölüm 2



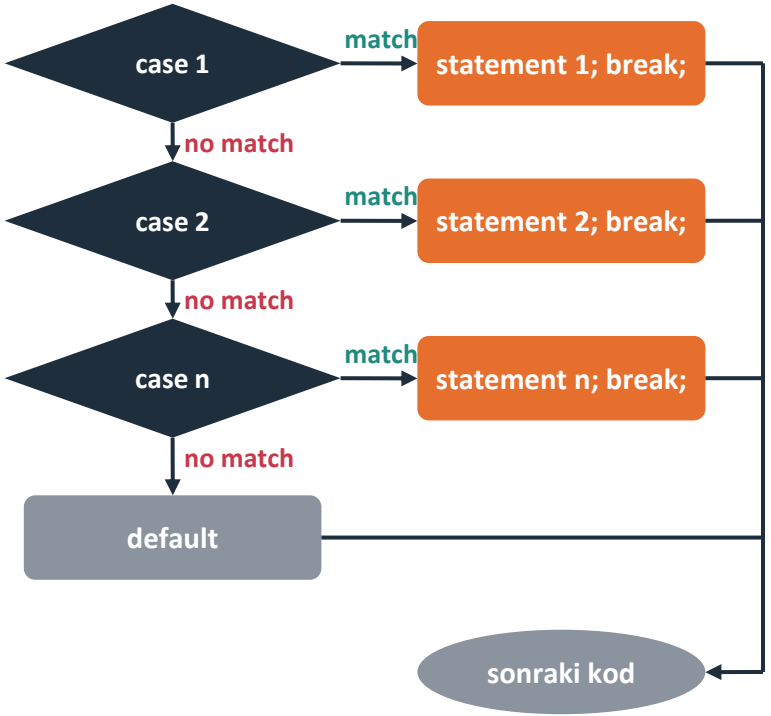
switch Statement

Bir değeri birçok sabitle karşılaştırmak

switch Statement: Syntax ve Akış

Syntax

```
switch (expression) {
    case value1:
        // value1 ile eşleşirse
        break;          // opsiyonel
    case value2:
        // value2 ile eşleşirse
        break;
    // ... istenildiği kadar case
    default:            // opsiyonel
        // hiçbir case eşleşmezse
}
```



switch Kuralları

İzin verilen tipler

- `byte`, `short`, `char`, `int` ve wrapper'ları
- `String` (Java 7+) ve `enum`

İzin verilmeyen tipler

- `long`, `float`, `double`, `boolean`
- Kaynakta `long`/`Long` destekleniyor deniyor; **bu yanlıştır**, compile error verir

case değerleri

- **Constant** (literal veya `final`) olmalıdır; variable olamaz
- **Unique** olmalıdır; tekrar eden case → compile error
- Tipi expression ile uyumlu olmalıdır

break ve default

- `break` yoksa sonraki case'ler de çalışır (**fall-through**)
- `default` opsiyoneldir; genelde en sona yazılır
- Aralık kontrolü yok: `case > 10` yazılamaz

switch Örneği: Ay Numarasından Ay Adı

```java
int month = 7;
String monthName;
switch (month) {
    case 1:  monthName = "Ocak";     break;
    case 2:  monthName = "Şubat";    break;
    case 3:  monthName = "Mart";     break;
    case 4:  monthName = "Nisan";    break;
    case 5:  monthName = "Mayıs";    break;
    case 6:  monthName = "Haziran";  break;
    case 7:  monthName = "Temmuz";   break;
    case 8:  monthName = "Ağustos";  break;
    case 9:  monthName = "Eylül";    break;
    case 10: monthName = "Ekim";     break;
    case 11: monthName = "Kasım";    break;
    case 12: monthName = "Aralık";   break;
    default: monthName = "Geçersiz ay!";
}
System.out.println(month + " - " + monthName);
```

Çıktı (Output)

```
7 - Temmuz
```

- Aynı variable'ı birçok sabitle karşılaştırırken switch, uzun bir if-else-if zincirinden daha okunaklıdır.
- **`default`** sayesinde **`monthName`** her durumda bir değer alır (aksi halde compile error).

Fall-Through: Hata mı, Özellik mi?

✗ break unutulursa

```
int number = 20;
switch (number) {
    case 10: System.out.println("10");
    case 20: System.out.println("20");
    case 30: System.out.println("30");
    default: System.out.println("Diğer");
}
```

Çıktı (Output)

```
20
30
Diğer
```

- Eşleşen case'ten sonra **break'e kadar** tüm satırlar çalışır.

✓ Bilinçli fall-through: case gruplama

```
char ch = 'O';
switch (Character.toLowerCase(ch)) {
    case 'a': case 'e': case 'ı': case 'i':
    case 'o': case 'ö': case 'u': case 'ü':
        System.out.println("Vowel (ünlü)");
        break;
    default:
        System.out.println("Consonant");
}
```

Çıktı (Output)

```
Vowel (ünlü)
```

- Kaynaktaki 10 ayrı case yerine gruplanmış hali.

switch ile String

```java
String levelString = "Expert";
int level;
switch (levelString) {
    case "Beginner":     level = 1; break;
    case "Intermediate": level = 2; break;
    case "Expert":       level = 3; break;
    default:             level = 0;
}
System.out.println("Your Level is: " + level);

String cmd = "EXIT";
switch (cmd.toLowerCase()) {     // case-sensitive!
    case "exit": System.out.println("Kapanıyor"); break;
    default:     System.out.println("Bilinmeyen komut");
}
```

Çıktı (Output)

```
Your Level is: 3
Kapanıyor
```

- String karşılaştırması **`equals()`** ile yapılır ve **büyük/küçük harfe duyarlıdır**.
- Expression **`null`** ise **`NullPointerException`** fırlatılır.
- String switch Java 7 ile geldi.

switch ile enum

```java
enum Day { SUN, MON, TUE, WED, THU, FRI, SAT }

public class Main {
    public static void main(String[] args) {
        for (Day d : Day.values()) {
            switch (d) {
                case SAT:
                case SUN:
                    System.out.println(d + ": Hafta sonu");
                    break;
                default:
                    System.out.println(d + ": Hafta içi");
            }
        }
    }
}
```

Çıktı (Output)

```
SUN: Hafta sonu
MON: Hafta içi
TUE: Hafta içi
WED: Hafta içi
THU: Hafta içi
FRI: Hafta içi
SAT: Hafta sonu
```

- case'lerde enum sabiti ön eksiz yazılır: **`case SAT`** (**`Day.SAT`** değil).
- **`Day.values()`** tüm sabitleri sırayla döndürür.
- Wrapper ile de çalışır: **`Integer age = 18; switch (age)`** → unboxing.

Modern switch (Java 14+): Arrow ve Expression

```java
int day = 6;
String type = switch (day) {  // switch expression
    case 1, 2, 3, 4, 5 -> "Hafta içi";
    case 6, 7          -> "Hafta sonu";
    default            -> "Geçersiz";
};
System.out.println(type);

char grade = 'B';
switch (grade) {  // arrow: fall-through yok
    case 'A'      -> System.out.println("Mükemmel");
    case 'B', 'C' -> System.out.println("İyi");
    default       -> System.out.println("Gelişmeli");
}
```

Çıktı (Output)

```
Hafta sonu
İyi
```

- **`->`** kullanıldığında **break gerekmez**, fall-through olmaz.
- Birden fazla label virgülle yazılır: **`case 6, 7`**.
- **switch expression** bir değer döndürür; çok satırlı block'ta **`yield`** kullanılır.

if-else-if mi, switch mi?

Kriter	if-else-if	switch
Koşul türü	Herhangi bir boolean ifade	Yalnızca eşitlik kontrolü
Desteklenen tipler	Tüm tipler	byte, short, char, int (+ wrapper), String, enum
Aralık (range) kontrolü	Var: x > 10 && x < 20	Yok (her değer ayrı case)
Okunabilirlik	Az sayıda koşulda iyi	Çok sayıda sabit değerde daha temiz
Performans	Koşullar sırayla kontrol edilir	JVM jump table kullanabilir
Tipik kullanım	Not aralıkları, karmaşık koşullar	Menü, gün/ay, komut seçimi

Bölüm 3



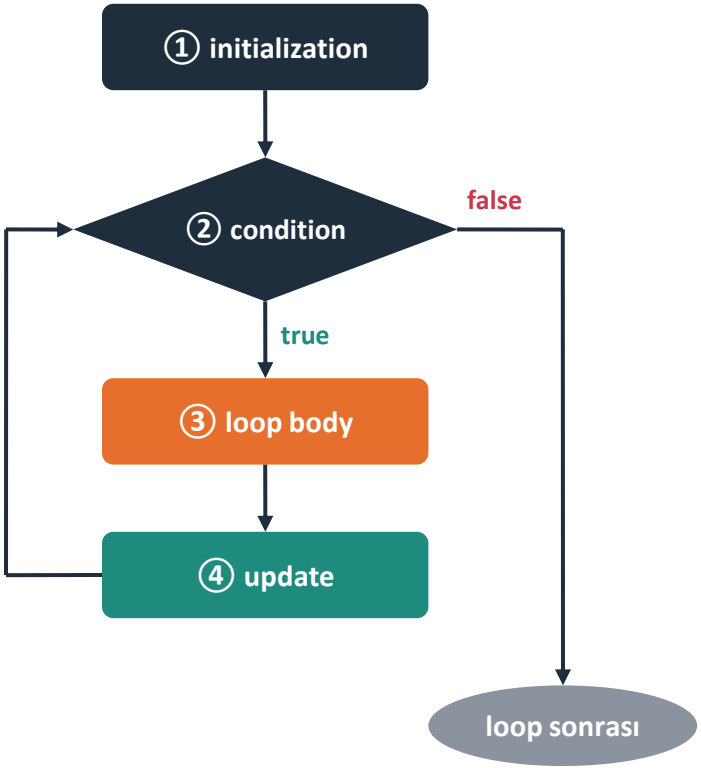
for Loop

Belirli sayıda tekrar

for Loop'un Anatomisi

```
for ( int i = 1 ; i <= 5 ; i++ ) { ③ loop body }
      ①          ②       ④
```

- ① **initialization**: bir kez, en başta çalışır.
- ② **condition**: her turdan önce kontrol edilir; false ise loop biter.
- ③ **body**: condition true ise çalışır.
- ④ **update**: her turun sonunda çalışır, sonra ②'ye dönülür.
- Sıra: ① → ② → ③ → ④ → ② → ③ → ④ → … → ② (false) → çıkış
- Tekrar sayısı **önceden biliniyorsa** for tercih edilir.



for Loop: Temel Örnekler

```java
for (int i = 1; i <= 10; i++) {
    System.out.print(i + " ");
}
System.out.println();

int number = 5, factorial = 1;
for (int i = 1; i <= number; i++) {
    factorial *= i;  // factorial = factorial * i
}
System.out.println(number + "! = " + factorial);

int sum = 0;
for (int i = 2; i <= 100; i += 2) {
    sum += i;
}
System.out.println("Çift sayılar toplamı: " + sum);
```

Çıktı (Output)

```
1 2 3 4 5 6 7 8 9 10
5! = 120
Çift sayılar toplamı: 2550
```

- **`print`** aynı satıra yazar, **`println()`** satır sonu ekler.
- Update kısmı yalnızca **`i++`** olmak zorunda değildir: **`i += 2`** ile 2'şer artış.
- Toplam/çarpım için **accumulator** variable loop'tan önce tanımlanır.

Trace: Factorial Loop'unu Adım Adım İzlemek

Tur	i	i <= 5 ?	factorial (önce)	factorial *= i (sonra)
1	1	true	1	1
2	2	true	1	2
3	3	true	2	6
4	4	true	6	24
5	5	true	24	120
—	6	false → loop biter	120	—

Trace table, loop'ların nasıl çalıştığını anlamanın ve hata ayıklamanın (debugging) en etkili yoludur. Öğrencilere kâğıt üzerinde trace yaptırın.

for Loop Varyasyonları

```java
for (int i = 10; i >= 1; i -= 3) {  // geriye doğru
    System.out.print(i + " ");
}
System.out.println();

for (int i = 0, j = 10; i < j; i++, j--) {  // iki variable
    System.out.print(i + "-" + j + " ");
}
System.out.println();

int k = 1;
for (; k < 100; ) {  // parçalar opsiyonel
    k *= 3;
}
System.out.println("k = " + k);

for (char c = 'a'; c <= 'e'; c++) {  // char ile
    System.out.print(c);
}
```

Çıktı (Output)

```
10 7 4 1
0-10 1-9 2-8 3-7 4-6
k = 243
abcde
```

- Init ve update kısmında **virgülle** birden fazla ifade yazılabilir.
- Üç kısmın hepsi opsiyoneldir; `for (;;)` sonsuz loop'tur.
- Loop variable'ı `char` dahil her tipten olabilir.

Nested for Loop: Çarpım Tablosu

```java
for (int i = 1; i <= 5; i++) {  // outer loop
    for (int j = 1; j <= 5; j++) {  // inner loop
        System.out.printf("%4d", i * j);
    }
    System.out.println();  // satır sonu
}
```

Çıktı (Output)

```
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
   4   8  12  16  20
   5  10  15  20  25
```

- Inner loop, outer loop'un **her turunda baştan sona** çalışır: toplam 5 × 5 = 25 çalışma.
- Outer loop **satırları**, inner loop **sütunları** üretir.
- **`printf("%4d", x)`**: sayıyı 4 karakter genişliğinde, sağa hizalı yazar.

Nested for ile Star Pattern'ler

Left-aligned triangle

```
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

Çıktı (Output)

```
*
* *
* * *
* * * *
* * * * *
```

- i. satırda i yıldız: inner loop sınırı **i**'ye bağlı.

Inverted triangle

```
int term = 5;
for (int i = 1; i <= term; i++) {
    for (int j = term; j >= i; j--) {
        System.out.print("* ");
    }
    System.out.println();
}
```

Çıktı (Output)

```
* * * * *
* * * *
* * *
* *
*
```

- Her satırda bir yıldız eksilir.

Algoritma Örnekleri: Asal Sayı ve Fibonacci

```java
int n = 29;
boolean isPrime = n > 1;
for (int i = 2; i * i <= n; i++) {
    if (n % i == 0) {
        isPrime = false;
        break;  // bölen bulundu, çık
    }
}
System.out.println(n + " asal mı? " + isPrime);

int a = 0, b = 1;
System.out.print("Fibonacci: ");
for (int i = 0; i < 10; i++) {
    System.out.print(a + " ");
    int next = a + b;
    a = b;
    b = next;
}
```

Çıktı (Output)

```
29 asal mı? true
Fibonacci: 0 1 1 2 3 5 8 13 21 34
```

- Asal kontrolünde **`i * i <= n`** yeterlidir: √n'den büyük bir bölen varsa, küçük bir eşi de vardır.
- **`break`** ile gereksiz turlar atlanır.
- Fibonacci'de her terim önceki iki terimin toplamıdır; iki variable "kaydırılarak" güncellenir.

Labeled for Loop ile break

break aa; → outer loop'u bitirir

```
aa:
for (int i = 1; i <= 3; i++) {
    bb:
    for (int j = 1; j <= 3; j++) {
        if (i == 2 && j == 2) {
            break aa;
        }
        System.out.println(i + " " + j);
    }
}
```

Çıktı (Output)

```
1 1   1 2   1 3
2 1
```

break bb; → sadece inner loop'u bitirir

```
aa:
for (int i = 1; i <= 3; i++) {
    bb:
    for (int j = 1; j <= 3; j++) {
        if (i == 2 && j == 2) {
            break bb;
        }
        System.out.println(i + " " + j);
    }
}
```

Çıktı (Output)

```
1 1   1 2   1 3
2 1
3 1   3 2   3 3
```

for Loop'ta Sık Yapılan Hatalar

Off-by-one hatası

- `i <= arr.length` → son turda `ArrayIndexOutOfBoundsException`
- Doğrusu: `i < arr.length` (index 0'dan başlar)

Loop sonrası noktalı virgül

- `for (int i = 0; i < 5; i++);`
- Body boş statement olur; alttaki block yalnızca **bir kez** çalışır

Yanlış yönde update

- `for (int i = 0; i < 10; i--)` → condition hep true, **sonsuz loop**
- Bilinçli sonsuz loop: `for (;;)`, durdurmak için Ctrl+C

Scope dışı kullanım

- `for (int i…)` içindeki `i`, loop bitince yok olur
- Loop sonrası gerekiyorsa variable'ı loop'tan önce tanımlayın

Bölüm 4



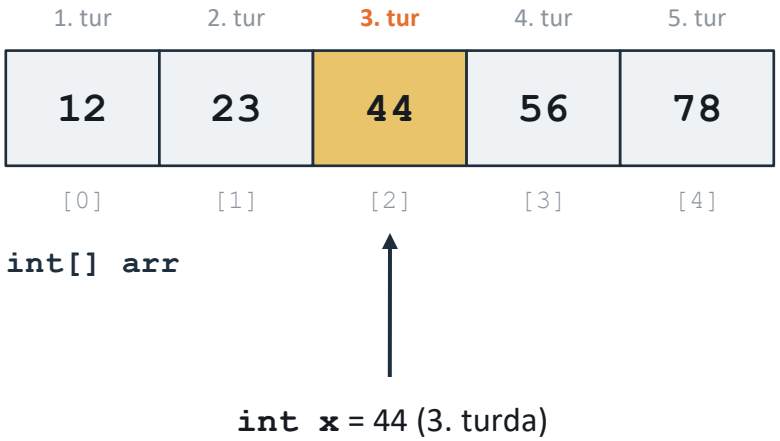
for-each Loop

Enhanced for: elemanları tek tek gezmek

for-each (Enhanced for) Loop

```
Syntax
for (dataType variable : arrayOrCollection) {
    // variable her turda sıradaki elemandır
}
```

- J2SE 5.0 (Java 5) ile geldi; array ve **Collection** (**`ArrayList`**, **`Set`**…) üzerinde çalışır.
- Index ve sayaç yönetimi gerekmez → **index-out-of-bounds hatası olmaz**.
- **`:`** işareti "içindeki her eleman için" diye okunur.



for-each: Array İşlemleri

```java
int[] arr = {12, 23, 44, 56, 78};
for (int x : arr) {
    System.out.print(x + " ");
}
System.out.println();

int total = 0, max = arr[0];
for (int x : arr) {
    total += x;
    if (x > max) {
        max = x;
    }
}
System.out.println("Toplam: " + total);
System.out.println("Maksimum: " + max);
double avg = (double) total / arr.length;
System.out.println("Ortalama: " + avg);
```

Çıktı (Output)

```
12 23 44 56 78
Toplam: 213
Maksimum: 78
Ortalama: 42.6
```

- Toplam, maksimum, minimum, ortalama, sayma gibi **okuma** işlemleri için idealdir.
- Ortalamada `(double)` cast'i integer division'ı önler.

for-each: Collection ve 2D Array

```java
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Ali");
        list.add("Ayşe");
        list.add("Mehmet");
        for (String name : list) {
            System.out.println(name + " (" + name.length() + ")");
        }

        int[][] matrix = {{1, 2, 3}, {4, 5, 6}};
        for (int[] row : matrix) {      // her satır bir array
            for (int v : row) {
                System.out.print(v + " ");
            }
            System.out.println();
        }
    }
}
```

Çıktı (Output)

```
Ali (3)
Ayşe (4)
Mehmet (6)
1 2 3
4 5 6
```

- **`ArrayList`**, **`HashSet`** gibi tüm **`Iterable`** yapılarla çalışır.
- 2D array'de dış loop'un variable'ı bir **satırdır** (**`int[]`**).

for-each'in Sınırlamaları

✗ for-each ile array değiştirilemez

```
int[] arr = {1, 2, 3};
for (int x : arr) {
    x = x * 10;  // sadece kopya değişir
}
System.out.println(Arrays.toString(arr));
```

Çıktı (Output)

```
[1, 2, 3]
```

- Loop variable'ı elemanın **kopyasıdır**.
- Index yok; ters yönde veya atlayarak gezilemez.

✓ Değiştirmek için klasik for

```
int[] arr = {1, 2, 3};
for (int i = 0; i < arr.length; i++) {
    arr[i] = arr[i] * 10;
}
System.out.println(Arrays.toString(arr));
```

Çıktı (Output)

```
[10, 20, 30]
```

- Index gerektiğinde klasik for kullanın.
- for-each sırasında Collection'dan eleman silmek → **`ConcurrentModificationException`**.

for mu, for-each mi?

Kriter	Klasik for	for-each
Index erişimi	Var (i)	Yok
Ters yönde / atlayarak gezme	Mümkün (i--, i += 2)	Mümkün değil
Array elemanını değiştirme	Mümkün (arr[i] = …)	Mümkün değil (kopya ile çalışır)
Index hatası riski	Var (off-by-one)	Yok
Okunabilirlik	Orta	Yüksek
Uygun olduğu yer	Sayaç, aralık, index gereken işler	Tüm elemanları okuma

Alıştırma: Çıktı Nedir?

1

Soru 1

```
int x = 15;
if (x > 10)
    if (x > 20) System.out.println("A");
    else System.out.println("B");
```

2

Soru 2

```
int n = 2;
switch (n) {
    case 1: System.out.print("1 ");
    case 2: System.out.print("2 ");
    case 3: System.out.print("3 ");
}
```

3

Soru 3

```
int count = 0;
for (int i = 0; i < 5; i++)
    for (int j = i; j < 5; j++)
        count++;
System.out.println(count);
```

4

Soru 4

```
int s = 0;
for (int v : new int[]{3, 1, 4, 1, 5})
    if (v % 2 == 1) s += v;
System.out.println(s);
```

Özet

if Statements
Koşul boolean olmalı; ladder'da sıra önemli; her zaman { } kullanın.

switch
Eşitlik kontrolü; long/float/double/boolean desteklenmez; break yoksa fall-through; Java 14+ arrow syntax.

for / for-each
for: sayaç ve index gereken işler; for-each: tüm elemanları okumak. Off-by-one hatalarına dikkat.

Sıradaki ders
while, do-while, break, continue ve comments.

Kaynak: tpointtech.com Java eğitim sayfaları (içerik genişletilmiş, örnekler eklenmiş ve hatalar düzeltilmiştir).